PF

LAB#09

NAME : ALIZA SHAMSHER

ROLL NO: 26K-2548

CLASS/SEC: BDS-1B

Problem 1: Sorting an Array

Untitled1.cpp

```c
#include <stdio.h>

void bubbleSort(int *arr, int n) {
    int i, j, temp;
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (*(arr + j) > *(arr + j + 1)) {
                temp = *(arr + j);
                *(arr + j) = *(arr + j + 1);
                *(arr + j + 1) = temp;
            }
        }
    }
}

void printArray(const int *arr, int n) {
    int i;
    for (i = 0; i < n; i++) {
        printf("%d ", *(arr + i));
    }
    printf("\n");
}

int main() {
```

```
26      int arr[20];
27
28      while (1) {
29          printf("enter number of scores: ");
30          scanf("%d", &n);
31          if (n >= 1 && n <= 20) {
32              break;
33          }
34
35          printf("Invalid size\n");
36      }
37
38      printf("Enter scores: ");
39      for (int i = 0; i < n; i++) {
40          scanf("%d", &arr[i]);
41      }
42
43      printf("before sorting: ");
44      printArray(arr, n);
45
46      bubbleSort(arr, n);
47
48      printf("after sorting: ");
49      printArray(arr, n);
50
51      return 0;
```

OUTPUT:

```
C:\Users\lenovo\Desktop\Untitled1.exe
enter number of scores: 5
Enter scores: 10 40 19 80 12
before sorting: 10 40 19 80 12
after sorting: 10 12 19 40 80

--------------------------------
Process exited after 13.43 seconds with return value 0
Press any key to continue . . .
```

Problem 2: Swap Prices

```c
#include <stdio.h>

void swapValues(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int a, b;

    printf("Enter price A: ");
    scanf("%d", &a);
    printf("Enter price B: ");
    scanf("%d", &b);

    printf("\nqBefore swap: A = %d", a);
     printf("Before swap: B = %d\n",  b);

    swapValues(&a, &b);

    printf("After swap: A = %d", a);
     printf("After swap:  B = %d\n",  b);

    return 0;
}
```

OUTPUT:

C:\Users\lenovo\Desktop\Untitled1.exe

```
Enter price A: 24
Enter price B: 90

qBefore swap: A = 24Before swap: B = 90
After swap: A = 90After swap:  B = 24

--------------------------------
Process exited after 4.566 seconds with return value 0
Press any key to continue . . .
```

Problem 3: Marks Analyzer

```c
#include <stdio.h>

int isValidMark(int m) {
    if (m >= 0 && m <= 100) {
        return 1;
    }
    return 0;
}

void findMinMax(int arr[], int size, int *min, int *max) {
    *min = arr[0];
    *max = arr[0];
    for (int i = 1; i < size; i++) {
        if (arr[i] < *min) {
            *min = arr[i];
        }
        if (arr[i] > *max) {
            *max = arr[i];
        }
    }
}

int main() {
    int marks[5];
    int min, max;

    for (int i = 0; i < 5; i++) {
        printf("Enter mark %d: ", i + 1);
        scanf("%d", &marks[i]);

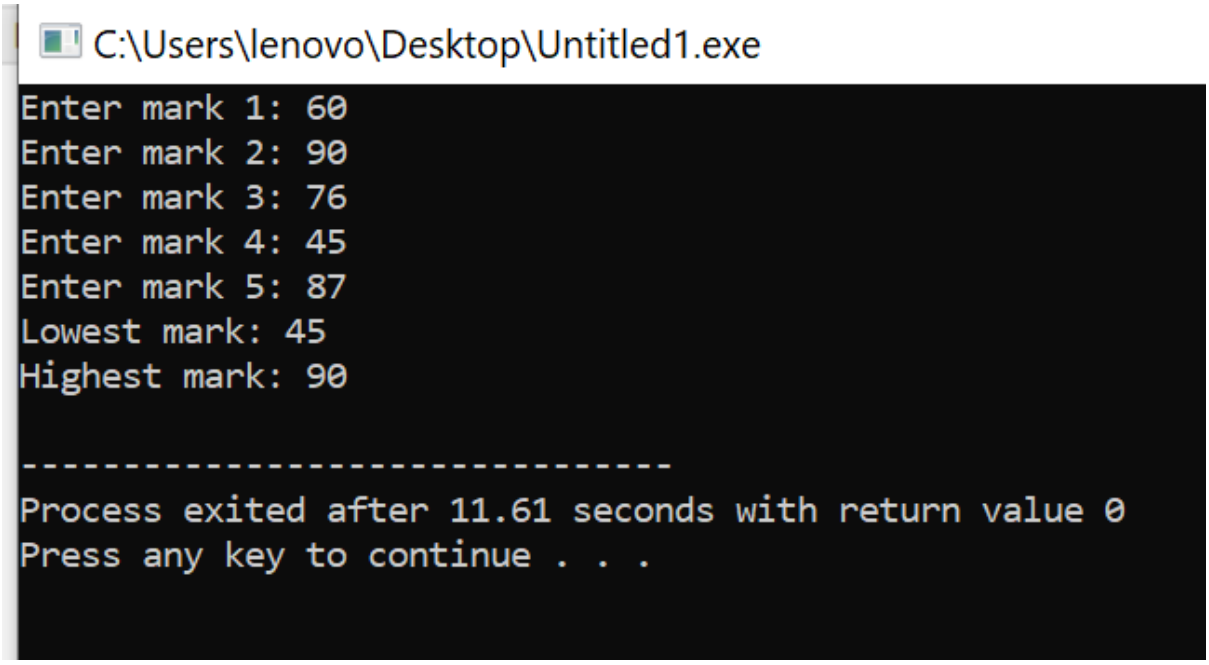
        while (!isValidMark(marks[i])) {
            printf("Invalid mark, enter again\n");
            printf("Enter mark %d: ", i + 1);
            scanf("%d", &marks[i]);
        }
    }

    findMinMax(marks, 5, &min, &max);

    printf("Lowest mark: %d\n", min);
    printf("Highest mark: %d\n", max);

    return 0;
}
```

OUTPUT:



Problem 4: Steps Tracker

```c
#include <stdio.h>

float averageSteps(int *steps, int n) {
    float sum = 0;
    for (int i = 0; i < n; i++) {
        sum += *(steps + i);
    }
    return sum / n;
}

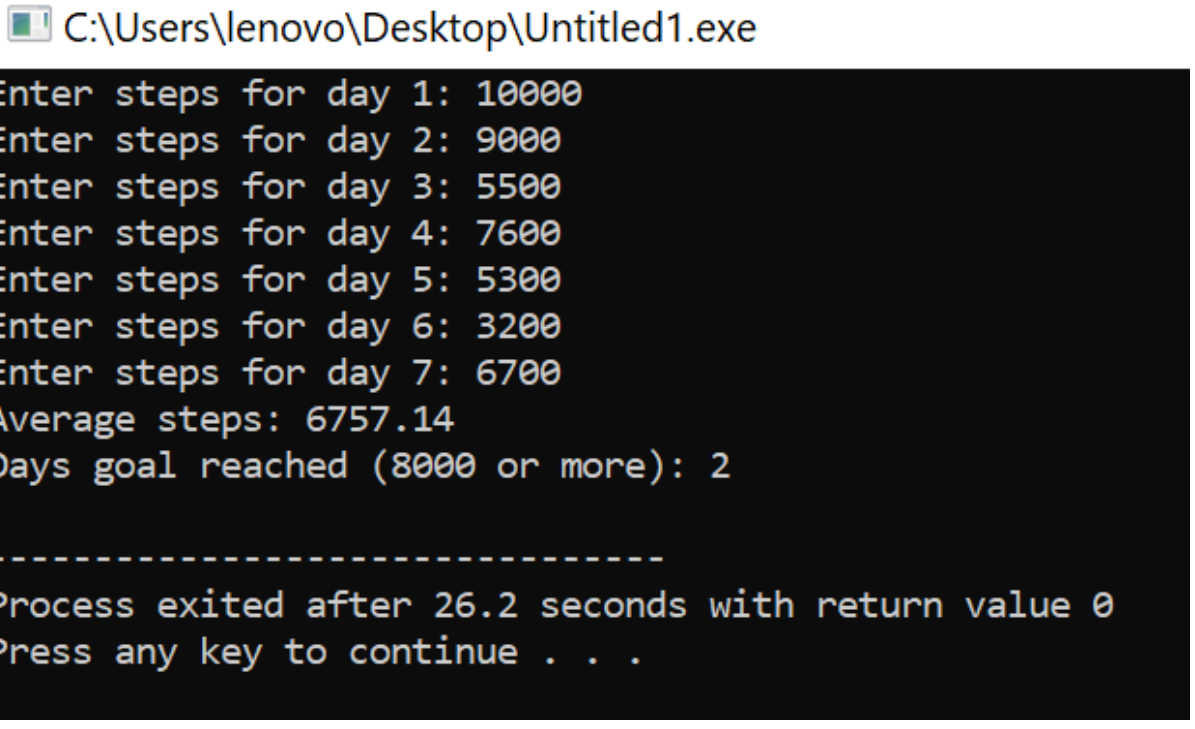
int countGoalDays(int *steps, int n, int goal) {
    int count = 0;
    for (int i = 0; i < n; i++) {
        if (*(steps + i) >= goal) {
            count++;
        }
    }
    return count;
}

int main() {
    int steps[7];

    for (int i = 0; i < 7; i++) {
        printf("Enter steps for day %d: ", i + 1);
        scanf("%d", &steps[i]);
```

```
26          scanf("%d", &steps[i]);
27      }
28
29      float avg = averageSteps(steps, 7);
30      int goalDays = countGoalDays(steps, 7, 8000);
31
32      printf("Average steps: %.2f\n", avg);
33      printf("Days goal reached (8000 or more): %d\n", goalDays);
34
35      return 0;
36  }
```

OUTPUT:



Problem 5: Game of Life

```c
#include <stdio.h>

void printGrid(int (*grid)[5]) {
    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 5; j++) {
            printf("%d ", grid[i][j] & 1);
        }
        printf("\n");
    }
}

int countNeighbours(int (*grid)[5], int row, int col) {
    int count = 0;
    for (int i = -1; i <= 1; i++) {
        for (int j = -1; j <= 1; j++) {
            if (i == 0 && j == 0) continue;
            int r = row + i;
            int c = col + j;
            if (r >= 0 && r < 5 && c >= 0 && c < 5) {
                if (grid[r][c] & 1) {
                    count++;
                }
            }
        }
    }
    return count;
}

void nextGeneration(int (*grid)[5]) {
    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 5; j++) {
            int live = countNeighbours(grid, i, j);
            int current = grid[i][j] & 1;
            int next = 0;
            if (current == 1 && (live == 2 || live == 3)) {
                next = 1;
            } else if (current == 0 && live == 3) {
                next = 1;
            }
            grid[i][j] |= (next << 1);
        }
    }
```

```
    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 5; j++) {
            grid[i][j] >>= 1;
        }
    }
}

int main() {
    int gens;
    int grid[5][5] = {0};

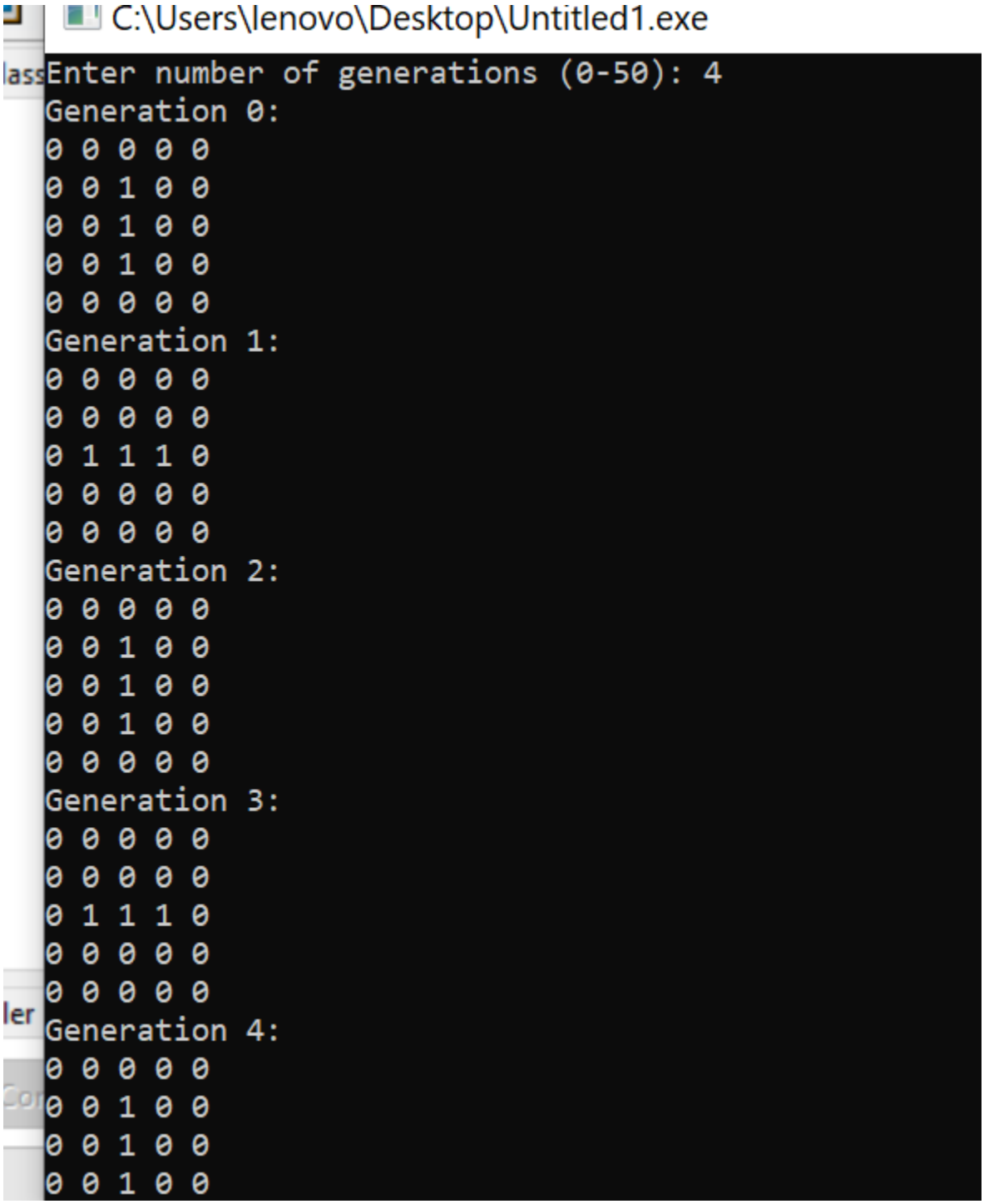
    grid[1][2] = 1;
    grid[2][2] = 1;
    grid[3][2] = 1;

    while (1) {
        printf("Enter number of generations (0-50): ");
        scanf("%d", &gens);
        if (gens >= 0 && gens <= 50) {
            break;
        }
        printf("Invalid number\n");
    }

    for (int g = 0; g <= gens; g++) {
        printf("Generation %d:\n", g);
        printGrid(grid);
        if (g < gens) {
            nextGeneration(grid);
        }
    }

    return 0;
}
```

OUTPUT:


C:\Users\lenovo\Desktop\Untitled1.exe
Enter number of generations (0-50): 4
Generation 0:
0 0 0 0 0
0 0 1 0 0
0 0 1 0 0
0 0 1 0 0
0 0 0 0 0
Generation 1:
0 0 0 0 0
0 0 0 0 0
0 1 1 1 0
0 0 0 0 0
0 0 0 0 0
Generation 2:
0 0 0 0 0
0 0 1 0 0
0 0 1 0 0
0 0 1 0 0
0 0 0 0 0
Generation 3:
0 0 0 0 0
0 0 0 0 0
0 1 1 1 0
0 0 0 0 0
0 0 0 0 0
Generation 4:
0 0 0 0 0
0 0 1 0 0
0 0 1 0 0
0 0 1 0 0

Problem 6: Bank Account

```c
#include <stdio.h>

int deposit(float *balance, float amount) {
    if (amount <= 0) {
        return 0;
    }
    *balance += amount;
    return 1;
}

int withdraw(float *balance, float amount) {
    if (amount <= 0 || amount > *balance) {
        return 0;
    }
    *balance -= amount;
    return 1;
}

int main() {
    float balance = 1000.0;
    int choice;
    float amount;

    do {
        printf("Choose option (1-4): ");
        scanf("%d", &choice);
```

```c
        switch (choice) {
            case 1:
                printf("Enter amount: ");
                scanf("%f", &amount);
                if (deposit(&balance, amount)) {
                    printf("Amount deposited successfully.\n");
                } else {
                    printf("Invalid amount.\n");
                }
                break;
            case 2:
                printf("Enter amount: ");
                scanf("%f", &amount);
                if (withdraw(&balance, amount)) {
                    printf("Amount withdrawn successfully.\n");
                } else {
                    printf("Insufficient funds or invalid amount.\n");
                }
                break;
            case 3:
                printf("Current balance: %.2f\n", balance);
                break;
            case 4:
                printf("Goodbye.\n");
```

```
51              printf("Goodbye.\n");
52              break;
53          default:
54              printf("Invalid option\n");
55              break;
56      }
57  } while (choice != 4);
58
59  return 0;
60 }
```

OUTPUT:

```
Choose option (1-4): 2
Enter amount: 1500
Insufficient funds or invalid amount.
Choose option (1-4): 1
Enter amount: 500
Amount deposited successfully.
Choose option (1-4): 3
Current balance: 1500.00
Choose option (1-4): 4
Goodbye.

--------------------------------
Process exited after 16.42 seconds with return value 0
Press any key to continue . . .
```

Problem 7: Reverse PlaylisT

```c
#include <stdio.h>

void reverseArray(int *arr, int n) {
    int *start = arr;
    int *end = arr + n - 1;
    int temp;
    while (start < end) {
        temp = *start;
        *start = *end;
        *end = temp;
        start++;
        end--;
    }
}

void printArray(const int *arr, int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", *(arr + i));
    }
    printf("\n");
}

int main() {
    int n;
    int arr[20];

```

```
27      while (1) {
28          printf("Enter number of songs: ");
29          if (scanf("%d", &n) != 1) {
30              break;
31          }
32          if (n >= 1 && n <= 20) {
33              break;
34          }
35          printf("Invalid size\n");
36      }
37
38      printf("Enter song IDs: ");
39      for (int i = 0; i < n; i++) {
40          scanf("%d", &arr[i]);
41      }
42
43      printf("Original: ");
44      printArray(arr, n);
45
46      reverseArray(arr, n);
47
48      printf("Reversed: ");
49      printArray(arr, n);
50
51      return 0;
52  }
```

OUTPUT:

```
C:\Users\lenovo\Desktop\Untitled3.exe
Enter number of songs: 5
Enter song IDs: 11 33 44 66 55
Original: 11 33 44 66 55
Reversed: 55 66 44 33 11

--------------------------------
Process exited after 10.26 seconds with return value 0
Press any key to continue . . .
```

Problem 8: Class Marks Table

```c
#include <stdio.h>

void computeTotals(int (*marks)[3], int rows, int totals[]) {
    for (int i = 0; i < rows; i++) {
        totals[i] = 0;
        for (int j = 0; j < 3; j++) {
            totals[i] += *(*(marks + i) + j);
        }
    }
}

int findTopper(const int *totals, int rows) {
    int maxIdx = 0;
    for (int i = 1; i < rows; i++) {
        if (*(totals + i) > *(totals + maxIdx)) {
            maxIdx = i;
        }
    }
    return maxIdx;
}

int main() {
    int marks[3][3];
    int totals[3];

    for (int i = 0; i < 3; i++) {
        printf("Enter marks for Student %d (3 subjects): ", i + 1);
        for (int j = 0; j < 3; j++) {
            scanf("%d", &marks[i][j]);
        }
    }

    computeTotals(marks, 3, totals);

    for (int i = 0; i < 3; i++) {
        printf("Total for Student %d: %d\n", i + 1, totals[i]);
    }

    int topper = findTopper(totals, 3);
    printf("Student %d is the topper with total %d\n", topper + 1, totals[topper]);

    return 0;
}
```

OUTPUT:

```
C:\Users\lenovo\Desktop\Untitled4.exe
Enter marks for Student 1 (3 subjects): 80 70 87
Enter marks for Student 2 (3 subjects): 67 68 90
Enter marks for Student 3 (3 subjects): 88 64 96
Total for Student 1: 237
Total for Student 2: 225
Total for Student 3: 248
Student 3 is the topper with total 248

--------------------------------
Process exited after 21.93 seconds with return value 0
Press any key to continue . . .
```

Problem 9: Library Shelves

```c
#include <stdio.h>

int sumShelf(const int *shelf, int count) {
    int sum = 0;
    for (int i = 0; i < count; i++) {
        sum += *(shelf + i);
    }
    return sum;
}

int findLargestShelf(const int *counts, int shelves) {
    int maxIdx = 0;
    for (int i = 1; i < shelves; i++) {
        if (*(counts + i) > *(counts + maxIdx)) {
            maxIdx = i;
        }
    }
    return maxIdx;
}

int main() {
    int shelfA[] = {101, 102, 103};
    int shelfB[] = {201, 202};
    int shelfC[] = {301, 302, 303, 304};

    int *shelves[] = {shelfA, shelfB, shelfC};
```

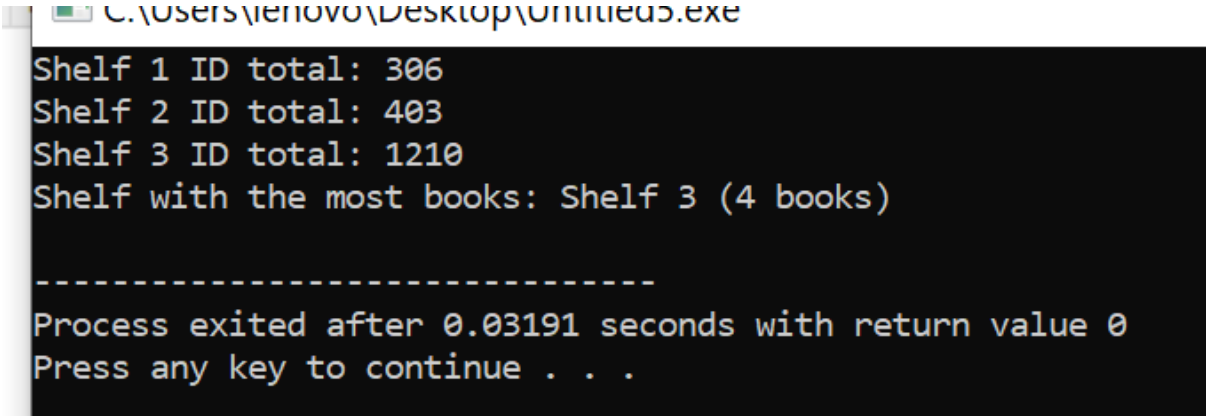
```c
    int counts[] = {3, 2, 4};

    for (int i = 0; i < 3; i++) {
        int total = sumShelf(shelves[i], counts[i]);
        printf("Shelf %d ID total: %d\n", i + 1, total);
    }

    int largest = findLargestShelf(counts, 3);
    printf("Shelf with the most books: Shelf %d (%d books)\n", largest + 1, counts[largest]);

    return 0;
}
```

OUTPUT:



Problem 10: Calculator with Function Pointer Table

```c
#include <stdio.h>

int add(int a, int b, int *result) {
    *result = a + b;
    return 1;
}

int subtract(int a, int b, int *result) {
    *result = a - b;
    return 1;
}

int multiply(int a, int b, int *result) {
    *result = a * b;
    return 1;
}

int divide(int a, int b, int *result) {
    if (b == 0) {
        return 0;
    }
    *result = a / b;
    return 1;
}

int main() {
    int (*ops[])(int, int, int *) = {add, subtract, multiply, divide};
    int choice, a, b, result;

    do {
        printf("Choice (1-5): ");
        scanf("%d", &choice);

        if (choice >= 1 && choice <= 4) {
            printf("Enter a and b: ");
            scanf("%d %d", &a, &b);

            if (ops[choice - 1](a, b, &result)) {
                if (choice == 1) printf("Add result: %d\n", result);
                else if (choice == 2) printf("Subtract result: %d\n", result);
                else if (choice == 3) printf("Multiply result: %d\n", result);
                else if (choice == 4) printf("Divide result: %d\n", result);
            } else {
                printf("Error: division by zero\n");
            }
        } else if (choice == 5) {
            printf("Goodbye.\n");
        } else {
            printf("Invalid choice\n");
        }
    } while (choice != 5);
```

```
51      } while (choice != 5);
52
53      return 0;
54  }
```

OUTPUT:

```
Choice (1-5): 4
Enter a and b: 10 0
Error: division by zero
Choice (1-5): 3
Enter a and b: 6 7
Multiply result: 42
Choice (1-5): 1
Enter a and b: 11 11
Add result: 22
Choice (1-5): 5
Goodbye.

--------------------------------
Process exited after 29.68 seconds with return value 0
Press any key to continue . . .
```

Problem 11: Generic Array Processor

```c
#include <stdio.h>

int square(int x) { return x * x; }
int negate(int x) { return -x; }
int isEven(int x) { return x % 2 == 0; }
int isPositive(int x) { return x > 0; }

void applyToArray(int *arr, int n, int (*fn)(int)) {
    for (int i = 0; i < n; i++) {
        arr[i] = fn(arr[i]);
    }
}

int countIf(const int *arr, int n, int (*pred)(int)) {
    int count = 0;
    for (int i = 0; i < n; i++) {
        if (pred(arr[i])) {
            count++;
        }
    }
    return count;
}

void printGeneric(void *data, char type, int n) {
    if (type == 'i') {
        int *p = (int *)data;
        for (int i = 0; i < n; i++) printf("%d ", p[i]);
        printf("\n");
    } else if (type == 'f') {
        float *p = (float *)data;
        for (int i = 0; i < n; i++) printf("%.2f ", p[i]);
        printf("\n");
    } else if (type == 'c') {
        char *p = (char *)data;
        for (int i = 0; i < n; i++) printf("%c ", p[i]);
        printf("\n");
    } else {
        printf("Invalid type '%c' for printGeneric\n", type);
    }
}

int main() {
    int n;
    int arr[50] = {3, -2, 8, -5, 6};

    while (1) {
        printf("Enter n (1-50): ");
        scanf("%d", &n);
        if (n >= 1 && n <= 50) {
            break;
        }
```

```
51          }
52          printf("Invalid size\n");
53      }
54
55      printf("Original: ");
56      printGeneric(arr, 'i', n);
57
58      applyToArray(arr, n, negate);
59      printf("After negate: ");
60      printGeneric(arr, 'i', n);
61
62      int posCount = countIf(arr, n, isPositive);
63      printf("Positive values after negate: %d\n", posCount);
64
65      int evenCount = countIf(arr, n, isEven);
66      printf("Even values after negate: %d\n", evenCount);
67
68      float floats[] = {1.50, 2.25};
69      printf("Float print (type f): ");
70      printGeneric(floats, 'f', 2);
71
72      printGeneric(arr, 'x', n);
73
74      return 0;
75  }
```

OUTPUT:

```
Enter n (1-50): 5
Original: 3 -2 8 -5 6
After negate: -3 2 -8 5 -6
Positive values after negate: 2
Even values after negate: 3
Float print (type f): 1.50 2.25
Invalid type 'x' for printGeneric

--------------------------------
Process exited after 1.386 seconds with return value 0
Press any key to continue . . .
```